

CS201: Database Systems — Indexing, Storage, and ACID Transactions

1. Relational Storage and Indexing Structures

Database management systems (DBMS) optimize data retrieval using specialized index data structures. The most prevalent indexing mechanism in relational engines (such as PostgreSQL and MySQL InnoDB) is the B+ Tree. Unlike standard binary search trees, B+ trees are self-balancing multi-way search trees with high branching factors (fan-out), ensuring minimal disk I/O operations. In a B+ Tree, all data records or leaf pointers reside exclusively in the leaf nodes, which are linked together sequentially to accelerate range queries. Internal nodes store only search keys and routing pointers.

2. Clustered vs. Secondary Indexes

A clustered index dictates the physical storage order of table records on disk; therefore, a table can possess only one clustered index (frequently the primary key). Secondary or non-clustered indexes contain secondary search keys paired with pointers (e.g., tuple identifiers or primary key values) that point to the actual table rows. Covering indexes include all columns requested by a specific SQL query, allowing the database query planner to satisfy the query entirely from index leaf nodes without accessing disk pages.

3. The ACID Transaction Model

Transactions are logical units of database work that must adhere to ACID properties: 1) Atomicity: all operations within a transaction succeed or all are rolled back, guaranteed using Write-Ahead Logging (WAL); 2) Consistency: transactions transition the database from one valid state to another according to integrity constraints; 3) Isolation: concurrent transactions execute without interfering with one another; 4) Durability: once committed, transaction results survive system crashes and power failures.

4. Transaction Isolation Levels and Anomalies

The SQL standard specifies four transaction isolation levels to balance consistency and concurrency performance: Read Uncommitted (allows dirty reads, non-repeatable reads, and phantom reads); Read Committed (prevents dirty reads by acquiring short-term read locks); Repeatable Read (prevents dirty and non-repeatable reads by holding read and write locks until commit time); and Serializable (highest level, eliminates all anomalies including phantom reads, typically enforced via Multi-Version Concurrency Control (MVCC) or Strict Two-Phase Locking (SS2PL)). A phantom read occurs when a transaction queries a range of rows twice and discovers new rows inserted by another committed transaction.